

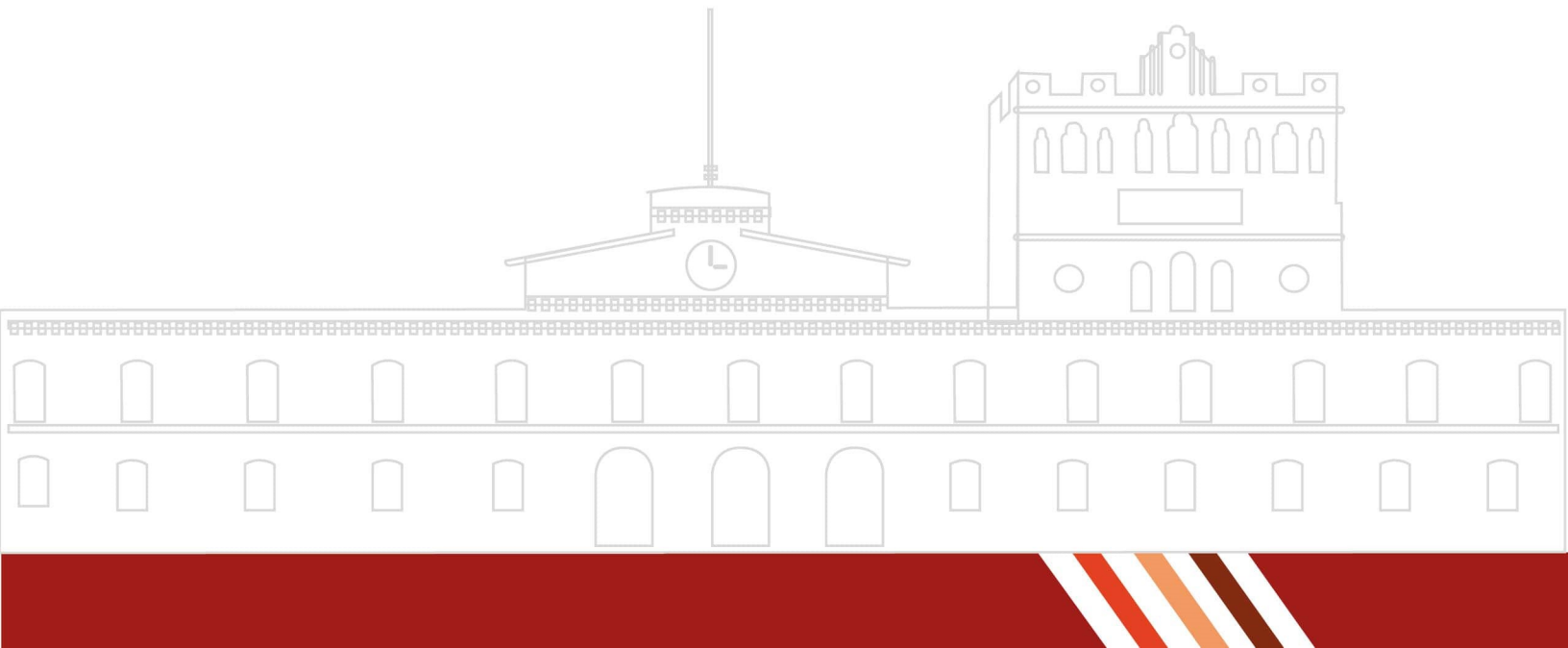
Reporte de Práctica 8

NOMBRE DE LA PRÁCTICA: Analizador Sintáctico

ALUMNOS:

- Cortés Santos María Fernanda
- Jácome Cayetano José David
- Palma Velazquez Ariadna Betsabé

Dr. Eduardo Cornejo-Velázquez



Introducción

El diseño y construcción de compiladores requiere la comprensión de fases fundamentales, siendo el análisis léxico y sintáctico los pilares para la interpretación de cualquier lenguaje estructurado. La práctica tiene como objetivo desarrollar un analizador léxico y sintáctico capaz de validar sentencias del Lenguaje de Consulta Estructurado (SQL), específicamente las operaciones del estándar DML (Data Manipulation Language): SELECT, INSERT, UPDATE y DELETE. Mediante el uso de herramientas para la generación de analizadores, se busca no solo reconocer la gramática correcta, sino también implementar mecanismos de recuperación de errores que permitan una ejecución continua y amigable para el usuario, demostrando así la aplicación práctica de la teoría de autómatas y lenguajes formales.

Marco teórico

Analizador Léxico

Es la primera fase de un compilador. Su función principal es leer los caracteres de entrada y agruparlos en secuencias lógicas con significado, llamadas tokens. Utiliza expresiones regulares y autómatas finitos deterministas (AFD) para identificar palabras clave, operadores e identificadores.

Analizador Sintáctico

Recibe los tokens generados por el analizador léxico y verifica que su orden y agrupación cumplan con las reglas gramaticales del lenguaje. Construye (conceptual o físicamente) un árbol de derivación sintáctica.

SQL (Structured Query Language)

Es el lenguaje estándar para la gestión de bases de datos relacionales. Sus sentencias se dividen en varias categorías, siendo el DML el encargado de la manipulación de datos mediante inserciones (INSERT), consultas (SELECT), actualizaciones (UPDATE) y eliminaciones (DELETE).

Recuperación de Errores (Modo Pánico)

Técnica utilizada en el análisis sintáctico donde, al detectar un error, el analizador descarta tokens de la entrada hasta encontrar un token de sincronización (como un salto de línea o un punto y coma), permitiendo reanudar el análisis sin detener la ejecución del programa.

Herramientas empleadas

- Lex (o Flex): Generador de analizadores léxicos. Se utilizó para definir las expresiones regulares que identifican los componentes léxicos del lenguaje SQL y retornar los tokens correspondientes.
- Yacc (o Bison): Generador de analizadores sintácticos (Parser). Se empleó para definir la gramática libre de contexto mediante reglas de producción y gestionar la lógica de validación.
- Lenguaje C: Lenguaje de programación estructurada utilizado como base subyacente para la ejecución lógica.

Desarrollo

El analizador léxico clasifica la entrada del usuario en distintas "clases" o tokens, los cuales son enviados a Yacc. Se definieron las siguientes familias léxicas (mapeadas a través de °/°union para capturar el valor en cadena):

- Palabras Reservadas: SELECT, FROM, WHERE, INSERT, INTO, VALUES, UPDATE, SET, DELETE, AND, OR.
- Identificadores (ID): Cadenas que inician con una letra y pueden contener números o guiones bajos, utilizados para nombres de tablas y columnas (ej. usuarios, id).
- Literales:
 - NUMBER: Secuencias de dígitos decimales.
 - STRING: Cadenas de texto encerradas entre comillas simples (ej. 'Laptop').
- Operadores Lógicos/Relacionales: EQ (=), LT (menor que), GT (mayor que).
- Símbolos y Puntuación: STAR (*), COMMA (,), PUNTOYCOMA (;), LPAREN ((), RPAREN ()), y ENDLINE (diagonalInvertida n para control de flujo).

Estructura Gramatical y Acciones Semánticas (Yacc)

El analizador sintáctico se desarrolló definiendo una gramática libre de contexto que modela la estructura de las sentencias SQL. La lógica y evaluación del código se fundamentan en los siguientes elementos clave:

- A diferencia de un analizador básico que solo retorna códigos numéricos, se implementó la directiva `%union` asignando el tipo de dato puntero a carácter (`char *`) para todos los elementos. Esto permite que los tokens y las reglas no solo validen la sintaxis, sino que transporten el valor semántico exacto (el texto capturado por la función `strdup` en Lex).
- La gramática fue diseñada de manera jerárquica y escalable. La evaluación principal recae en la regla `input`, que itera sobre cada línea de texto utilizando el token de control `ENDLINE`. A partir de la regla `query`, la evaluación se ramifica limpiamente hacia las cuatro operaciones soportadas, facilitando el mantenimiento del código.
- Cada regla gramatical está acompañada de bloques de código en C que se ejecutan al encontrar una producción válida. Haciendo uso de las pseudovariantes posicionales de Yacc (`SignoPesos1`, `SignoPesos2`, `SignoPesos3`, etc.), el sistema es capaz de interceptar los valores y mostrarlos dinámicamente en la interfaz de usuario.

Orden de Compilación

Para construir el analizador y generar el ejecutable final, se utilizó un entorno de desarrollo con herramientas integradas, lo que permite abstraer los comandos de consola en una interfaz gráfica. El proceso de construcción se llevó a cabo siguiendo una estricta secuencia utilizando los iconos de la barra de herramientas.

1. Generación del Código Léxico (Archivo `.l`): se inicializó el proceso haciendo clic en el botón 1 (icono de llave inglesa). Esta acción compila las expresiones regulares y genera automáticamente el archivo fuente en C subyacente (`lex.yy.c`).
2. Generación Sintáctica y Construcción del Ejecutable (Archivo `.y`): para compilar la gramática y vincular correctamente ambas fases, se ejecutó una secuencia de tres pasos en la barra de herramientas:
 - Se hizo clic en el icono número 3 para compilar las reglas de producción de Yacc. Esto genera tanto el código en C (`y.tab.c`) como el archivo de cabecera (`y.tab.h`) necesario para que el analizador léxico reconozca los tokens.
 - A continuación, se presionó el icono número 4 para compilar los archivos generados en C (tanto el léxico como el sintáctico).
 - Finalmente, se hizo clic en el icono número 5. Esta acción se encarga de enlazar (linkear) los archivos objeto y construir el archivo ejecutable `.exe` del sistema.

Capacidades del Sistema

- Es capaz de procesar correctamente las cuatro sentencias principales de SQL, desglosando en tiempo real las columnas solicitadas, las tablas destino y las condiciones aplicadas
- Gracias al uso de `°/°union` y `strdup`, el sistema extrae el valor exacto del texto introducido por el usuario y lo utiliza en la lógica de Yacc, no solo reconociendo la estructura, sino los datos en sí.
- El analizador no se cierra ante entradas inválidas (como 456378 o secuencias de letras sin sentido). El uso de la regla error `ENDLINE` limpia el buffer y reporta "ANALIZADOR RECUPERADO", listo para la siguiente línea.
- Logra diferenciar entre una sentencia totalmente estructurada pero incompleta (ej. olvidar el ; al final) reportando un "ERROR FALTA PUNTO Y COMA", frente a un error de gramática general.

Posibles Áreas de Mejora

- Actualmente, el analizador interactúa exclusivamente a través de la entrada y salida estándar (consola interactiva).

Código

- Archivo.l https://github.com/MaFecs/Actividades_Automatas_y_Compiladores/blob/89ddac29ce3c49fdcce44eProyectoFinal/ProyectoFinal.l
- Archivo.y https://github.com/MaFecs/Actividades_Automatas_y_Compiladores/blob/89ddac29ce3c49fdcce44eProyectoFinal/ProyectoFinal.y

Resultados

Pruebas válidas

```
Escribe tus sentencias SQL:
SELECT * FROM usuarios;
COLUMNA: [TODAS] (*)
TABLA: usuarios
OPERACION: Consulta (SELECT)
ESTADO: SENTENCIA SQL VALIDA

SELECT nombre, edad FROM empleados WHERE edad > 18;
COLUMNA: nombre
COLUMNA: edad
TABLA: empleados
CLAUSULA: WHERE DETECTADO
CONDICION: edad > 18
OPERACION: Consulta (SELECT)
ESTADO: SENTENCIA SQL VALIDA

INSERT INTO productos VALUES (101, 'Laptop');
TABLA DESTINO: productos
VALOR NUMERICO: 101
VALOR TEXTO: 'Laptop'
OPERACION: Insercion (INSERT)
ESTADO: SENTENCIA SQL VALIDA

INSERT INTO clientes (nombre, correo) VALUES ('Juan', 'juan@mail.com');
TABLA DESTINO: clientes
COLUMNA: nombre
COLUMNA: correo
VALOR TEXTO: 'Juan'
VALOR TEXTO: 'juan@mail.com'
OPERACION: Insercion (INSERT)
ESTADO: SENTENCIA SQL VALIDA

UPDATE inventario SET stock = 50 WHERE id = 5 AND tipo = 'hardware';
ACTUALIZANDO TABLA: inventario
VALOR NUMERICO: 50
ASIGNACION: stock = 50
CLAUSULA: WHERE DETECTADO
VALOR NUMERICO: 5
CONDICION: id = 5
LOGICA: AND
VALOR TEXTO: 'hardware'

CONDICION: edad > 18
OPERACION: Consulta (SELECT)
ESTADO: SENTENCIA SQL VALIDA

INSERT INTO productos VALUES (101, 'Laptop');
TABLA DESTINO: productos
VALOR NUMERICO: 101
VALOR TEXTO: 'Laptop'
OPERACION: Insercion (INSERT)
ESTADO: SENTENCIA SQL VALIDA

INSERT INTO clientes (nombre, correo) VALUES ('Juan', 'juan@mail.com');
TABLA DESTINO: clientes
COLUMNA: nombre
COLUMNA: correo
VALOR TEXTO: 'Juan'
VALOR TEXTO: 'juan@mail.com'
OPERACION: Insercion (INSERT)
ESTADO: SENTENCIA SQL VALIDA

UPDATE inventario SET stock = 50 WHERE id = 5 AND tipo = 'hardware';
ACTUALIZANDO TABLA: inventario
VALOR NUMERICO: 50
ASIGNACION: stock = 50
CLAUSULA: WHERE DETECTADO
VALOR NUMERICO: 5
CONDICION: id = 5
LOGICA: AND
VALOR TEXTO: 'hardware'
CONDICION: tipo = 'hardware'
OPERACION: Actualizacion (UPDATE)
ESTADO: SENTENCIA SQL VALIDA

DELETE FROM bitacora WHERE fecha < 2024;
BORRANDO DE TABLA: bitacora
CLAUSULA: WHERE DETECTADO
CONDICION: fecha < 2024
OPERACION: Eliminacion (DELETE)
ESTADO: SENTENCIA SQL VALIDA
```

Pruebas inválidas

```
C:\Users\Lenovo\Documents\ X + v

SELECT * FROM alumnos
  COLUMNA: [TODAS] (*)
  TABLA: alumnos
OPERACION: Consulta (SELECT)
ERROR FALTA PUNTO Y COMA

DELETE FROM reportes WHERE id = 10
BORRANDO DE TABLA: reportes
CLAUSULA: WHERE DETECTADO
  VALOR NUMERICO: 10
  CONDICION: id = 10
OPERACION: Eliminacion (DELETE)
ERROR FALTA PUNTO Y COMA

UPDATE cuenta SET saldo = 100
ACTUALIZANDO TABLA: cuenta
  VALOR NUMERICO: 100
  ASIGNACION: saldo = 100
OPERACION: Actualizacion (UPDATE)
ERROR FALTA PUNTO Y COMA

SELECT nombre usuarios;
  COLUMNA: nombre
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

SELECT * FROM tabla WHERE > 10 id;
  COLUMNA: [TODAS] (*)
  TABLA: tabla
  CLAUSULA: WHERE DETECTADO
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

UPDATE mi_tabla valor = 10 WHERE id = 1;
  ACTUALIZANDO TABLA: mi_tabla
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

INSERT INTO tabla (1, 'error');
  TABLA DESTINO: tabla
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

WHERE SELECT FROM tabla;
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

C:\Users\Lenovo\Documents\ X + v

Escribe tus sentencias SQL:
456378
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

jadncmwywjnd;
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

8sjs7jsl0dmcy;
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

82635djd9282
ERROR DE SINTAXIS: syntax error
ANILIZADOR RECUPERADO

|
```

Conclusiones

La práctica demuestra con éxito la colaboración entre el análisis léxico y sintáctico utilizando herramientas de generación automática. Se comprobó que estructurar un programa bajo un enfoque modular (separando el reconocimiento de patrones de la lógica de evaluación gramatical) reduce drásticamente la complejidad del código comparado con un diseño hecho desde cero. La incorporación de la recuperación de errores resultó ser una característica vital, simulando el comportamiento robusto que se espera de intérpretes comerciales y consolas de bases de datos reales.

Referencias Bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2a. ed.). Pearson Educación.
- Levine, J. R., Mason, T., y Brown, D. (1992). *Lex y Yacc* (2nd ed.). O'Reilly Media.